# 7 Algorithm design and problem solving

**In this chapter, you will learn about:**

- ★ the stages in the program development cycle:
  - analysis
  - design
  - coding
  - testing
- ★ computer systems and sub-systems
- ★ problem decomposition into component parts
- ★ methods used to design and construct solutions to problems
- ★ the purpose of an algorithm and the processes involved in it
- ★ standard methods of solution:
  - linear search
  - bubble sort
  - totalling
  - counting
  - finding average, maximum, minimum
- ★ validation checks when data is input
- ★ verification checks when data is input
- ★ use of different types of test data including:
  - documentation of a dry run using a trace table
- ★ writing, amending, identifying, and correcting errors in:
  - flowcharts
  - programs
  - pseudocode.

## 7.1 The program development life cycle

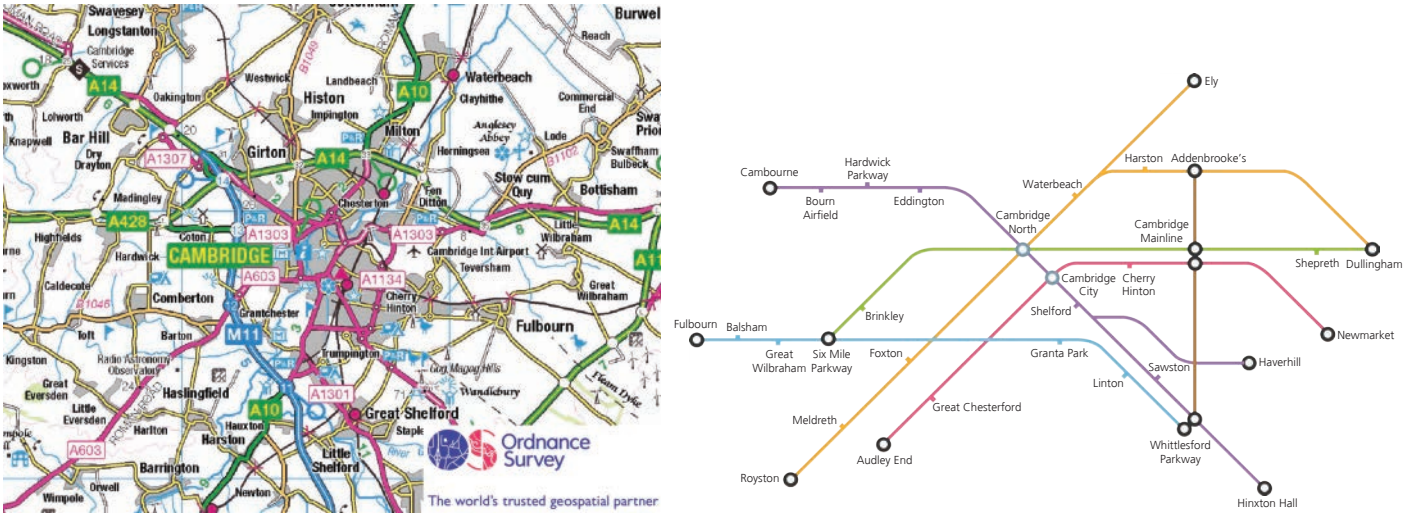
The program development life cycle is divided into five stages: analysis, design, coding, testing and maintenance. This chapter and Chapter 8 will discuss the four stages listed below:

- » **analysis**
- » **design**
- » **coding**
- » **testing**

### 7.1.1 Analysis

Before any problem can be solved, it needs to be clearly defined and set out so anyone working on the solution understands what is needed. This is called the 'requirements specification' for the program. The analysis stage uses **abstraction** and **decomposition** tools to identify exactly what is required from the program.

Abstraction keeps the key elements required for the solution to the problem and discards any unnecessary details and information that is not required. For example, a map only shows what is required for travelling from one place to another. Different methods of transport will require different types of map.




▲ **Figure 7.1** Road map and rail map

**Find out more**

Decompose getting dressed further, it can get quite complicated to show all the details required.

Decomposition breaks down a complex problem into smaller parts, which can then be subdivided into even smaller parts, that can be solved easily. Any daily task can be divided into its constituent parts.

For example, getting dressed:

- Select items to wear
- Remove any clothes being worn
- Put selected items on in order.

## 7.1.2 Design

The program specification from the analysis stage is used to show to how the program should be developed. When the design stage is complete, the programmer should know what is to be done, i.e. all the tasks that need to be completed, how each task is to be performed and how the tasks work together. This can be formally documented using **structure charts**, **flowcharts** and **pseudocode** – see Section 7.2.

## 7.1.3 Coding and iterative testing

The program or set of programs is developed. Each module of the program is written using a suitable programming language and then tested to see if it works. **Iterative testing** means that modular tests are conducted, code amended, and tests repeated until the module performs as required.

## 7.1.4 Testing

The completed program or set of programs is run many times with different sets of test data. This ensures that all the tasks completed work together as specified in the program design.

# 7.2 Computer systems, sub-systems and decomposition

A **computer system** is made up of software, data, hardware, communications and people; each computer system can be divided up into a set of sub-systems. Each sub-system can be further divided into sub-systems and so on until each sub-system just performs a single action.

Computer systems can be very large, very small or any size in between; most people interact with many different computer systems during their daily life without realising it.

**Find out more**

Find at least **five** computer systems you frequently use in your daily life; see if you can decide the size of each system.

For example, when you wake up in the morning, you might use an app on your smartphone for your alarm, then you might check the weather forecast on your computer before driving to work.

The alarm program is a very small computer system but when you check the weather forecast, you obtain the information you need from one of the largest computer systems in the world.

## 7.2.1 The computer system and its sub-systems

In order to understand how a computer system is built up and how it works it is often divided up into sub-systems. This division can be shown using top-down design to produce structure diagrams that demonstrate the modular construction of the system. Each sub-system can be developed by a programmer as a sub-routine. How each sub-routine works can be shown by using **flowcharts** or **pseudocode**.

**Top-down design** is the decomposition of a computer system into a set of sub-systems, then breaking each sub-system down into a set of smaller sub-systems, until each sub-system just performs a single action. This is an effective way of designing a computer system to provide a solution to a problem, since each part of the problem is broken down into smaller more manageable problems. The process of breaking down into smaller sub-systems is called stepwise refinement.

This structured approach works for the development of both large and small computer systems. When larger computer systems are being developed this means that several programmers can work independently to develop and test different sub-systems for the same system at the same time. This reduces the development and testing time.

## 7.2.2 Decomposing a problem

Any problem that uses a computer system for its solution needs to be decomposed into its component parts. The component parts of any computer system are:

- **inputs** – the data used by the system that needs to be entered while the system is active
- **processes** – the tasks that need to be performed using the input data and any other previously stored data
- **outputs** – information that needs to be displayed or printed for the users of the system
- **storage** – data that needs to be stored in files on an appropriate medium for use in the future.

## Example 1: An alarm app

For example, the alarm app can be decomposed into:

- inputs – time to set the alarm, remove a previously set alarm time, switch an alarm off, press snooze button
- processes – continuously check if the current time matches an alarm time that has been set, storage and removal of alarm times, management of snooze
- outputs – continuous sound/tune (at alarm time or after snooze time expired)
- storage – time(s) for alarms set.

## Activity 7.1

Using one of the computer systems that you identified, decompose it into its component parts of inputs, processes, outputs and storage.

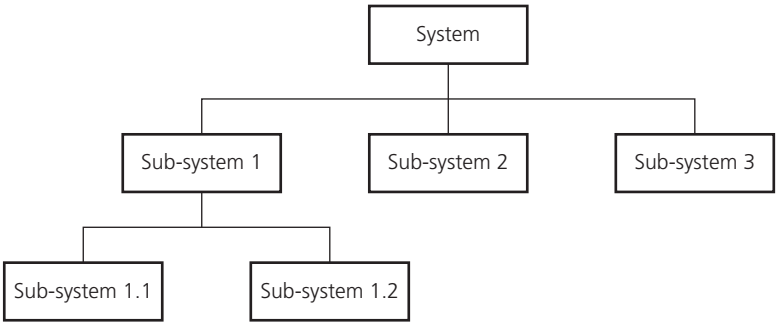
## 7.2.3 Methods used to design and construct a solution to a problem

Solutions to problems need to be designed and developed rigorously. The use of formal methods enables the process to be clearly shown for others to understand the proposed solution. The following methods need to be used by IGCSE Computer Science students:

- structure diagrams
- flowcharts
- pseudocode.

### Structure diagrams

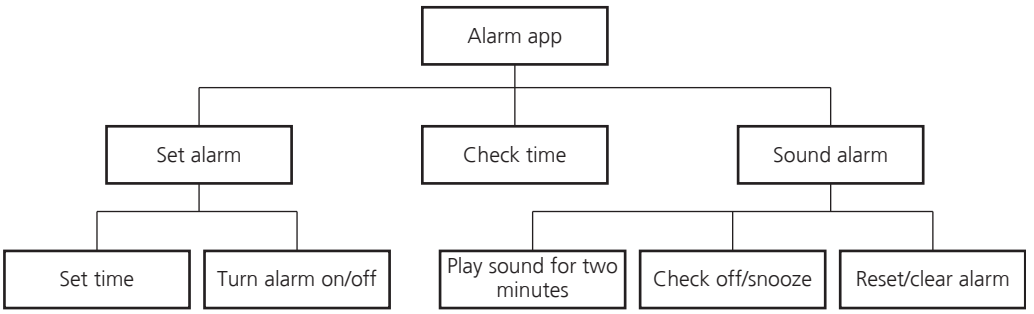
Structure diagrams can be used to show top-down design in a diagrammatic form. **Structure diagrams** are hierarchical, showing how a computer system solution can be divided into sub-systems with each level giving a more detailed breakdown. If necessary, each sub-system can be further divided.




▲ **Figure 7.2** Basic structure diagram

## Example 2: Alarm app for a smart phone

Consider the alarm app computer system for a smart phone; this could be divided into three sub-systems, setting the alarm, checking for the alarm time, sounding the alarm. These sub-systems could then be further sub-divided; a structure diagram makes the process clearer.




▲ **Figure 7.3** Structure diagram for alarm app

### Find out more

Draw a structure diagram for cleaning your teeth. If you are brave enough ask another student to try out the system to see if it works.

## Activity 7.2

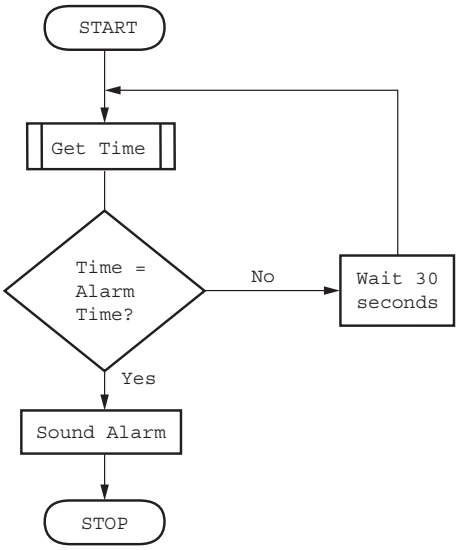
Break down the 'Check time' sub-system from the smart phone alarm app into further sub-systems.

### Flowcharts

A **flowchart** shows diagrammatically the steps required to complete a task and the order that they are to be performed. These steps, together with the order, are called an **algorithm**. Flowcharts are an effective way to communicate how the algorithm that makes up a system or sub-system works.

## Example 3: Checking for the alarm time

Have a look at a flowchart showing how the checking for the alarm time sub-system works.

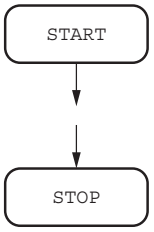



◀ **Figure 7.4** Flowchart for check time sub-system

Flowcharts are drawn using standard flowchart symbols.

### Begin/End

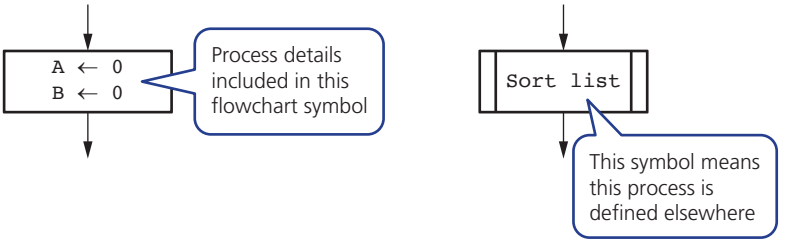
**Terminator flowchart symbols** are used at the beginning and end of each flowchart.




▲ **Figure 7.5** Terminator symbols

### Process

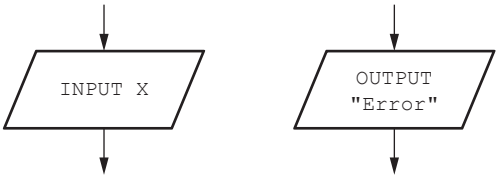
**Process flowchart symbols** are used to show actions, for example, when values are assigned to variables. If a process has been defined elsewhere then the name of that process is shown.




▲ **Figure 7.6** Process symbols

### Input and output

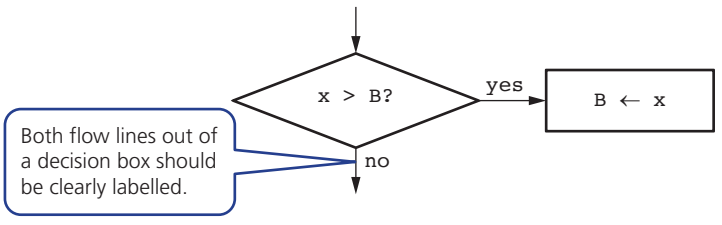
The same flowchart symbol is used to show the input of data and output of information.




▲ **Figure 7.7** Symbol used to show input and symbol used to show output

### Decision

**Decision flowchart symbols** are used to decide which action is to be taken next; these can be used for selection and repetition/iteration. There are always two outputs from a decision flowchart symbol.




▲ **Figure 7.8** Decision symbol

▲ **Figure 7.9** Flow line

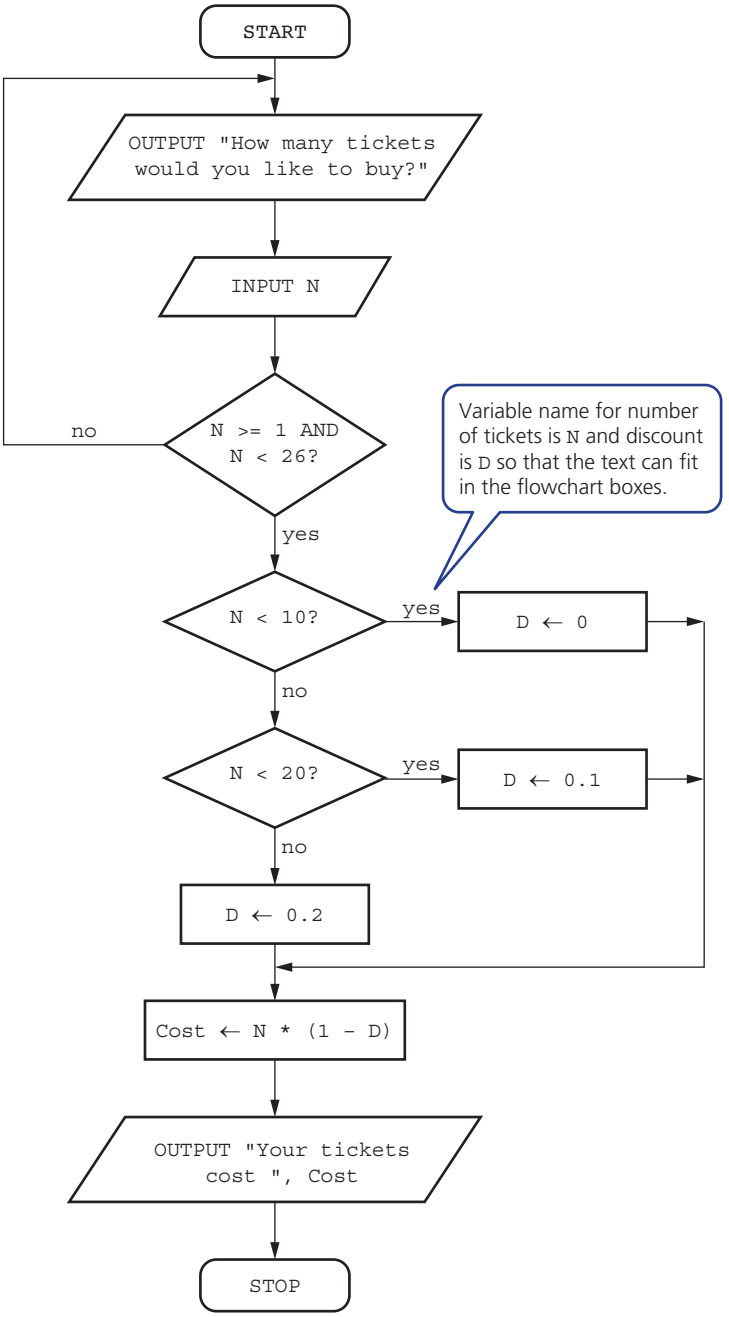
### Flow lines

**Flowchart flow lines** use arrows to show the direction of flow, which is usually, but not always, top to bottom and left to right.

## Example 4: Concert ticket sales

Tickets are sold for a concert at $20 each, if 10 tickets are bought then the discount is 10%, if 20 tickets are bought the discount is 20%. No more than 25 tickets can be bought in a single transaction.

This is flowchart showing an algorithm to calculate the cost of buying a given number of tickets:




▲ **Figure 7.10** Flowchart for ticket cost calculator

## Pseudocode

**Pseudocode** is a simple method of showing an algorithm. It describes what the algorithm does by using English key words that are very similar to those used in a high-level programming language. Data items to be processed by the algorithm are given meaningful names in the same way that variables and constants are in a high-level programming language. However, pseudocode is not bound by the strict syntax rules of a programming language. It does what its name says, it pretends to be programming code!

To ensure that pseudocode is easily understandable by others it is useful to be consistent in the way that it is written.

The pseudocode in this book is written in the following way to match the pseudocode given in the IGCSE Computer Science syllabus and to help you understand the algorithms more easily:

- a non-proportional font is used throughout
- all keywords (words used to describe a specific action e.g. INPUT) are written in capital letters
- all names given to data items and subroutines start with a capital letter
- where conditional and loop statements are used, repeated or selected statements are indented by two spaces.

### The pseudocode for an assignment statement

A value is assigned to an item/variable using the ← operator. The variable on the left of the ← is assigned the value of the expression on the right. The expression on the right can be a single value or several values combined with any of the following mathematical operators.

**Table 7.1** Mathematical operators

| Operator | Action |
|---|---|
| + | Add |
| – | Subtract |
| * | Multiply |
| / | Divide |
| ^ | Raise to the power |
| ( ) | Group |

Examples of pseudocode assignment statements:

| | |
|---|---|
| `Cost ← 10` | **Cost** has the value 10 |
| `Price ← Cost * 2` | **Price** has the value 20 |
| `Tax ← Price * 0.12` | **Tax** has the value 2.4 |
| `SellingPrice ← Price + Tax` | **SellingPrice** has the value 22.4 |
| `Gender ← "M"` | **Gender** has the value **M** |
| `Chosen ← False` | **Chosen** has the value **False** |

## Activity 7.3

What values will the following variables have after the assignments have been completed?

```
Amount ← 100
TotalPrice ← Amount * 3.5
Discount ← 0.2
FinalPrice ← TotalPrice - TotalPrice * Discount
Name ← "Nikki"
Message ← "Hello " + Name
```

### The pseudocode for conditional statements

When different actions are performed by an algorithm according to the values of the variables, conditional statements can be used to decide which action should be taken.

There are two types of conditional statement:

1 a condition that can be true or false such as: **IF … THEN … ELSE … ENDIF**

```
IF Age < 18
  THEN
    OUTPUT "Child"
  ELSE
    OUTPUT "Adult"
ENDIF
```

2 a choice between several different values, such as: **CASE OF … OTHERWISE … ENDCASE**

```
CASE OF Grade
  "A" : OUTPUT "Excellent"
  "B" : OUTPUT "Good"
  "C" : OUTPUT "Average"
  OTHERWISE OUTPUT "Improvement is needed"
ENDCASE
```

### IF … THEN … ELSE … ENDIF

For an **IF** condition the **THEN** path is followed if the condition is true and the **ELSE** path is followed if the condition is false. There may or may not be an **ELSE** path. The end of the statement is shown by **ENDIF**.

There are different ways that an **IF** condition can be set up:

- use of a Boolean variable that can have the value **TRUE** or **FALSE** (see Chapter 8 for details of Boolean variables). For example:

```
IF Found
  THEN
    OUTPUT "Your search was successful"
  ELSE
    OUTPUT "Your search was unsuccessful"
ENDIF
```

- comparisons made by using comparison operators, where comparisons are made from left to right, for example: **A > B** means '**A** is greater than **B**' Comparisons can be simple or more complicated, for example:

```
IF ((Height > 1) OR (Weight > 20)) AND (Age < 70) AND
  (Age > 5)
  THEN
    OUTPUT "You can ride"
  ELSE
    OUTPUT "Too small, too young or too old"
ENDIF
```

**Table 7.2** Comparison operators

| Operator | Comparison |
|---|---|
| > | Greater than |
| < | Less than |
| = | Equal |
| >= | Greater than or equal |
| <= | Less than or equal |
| <> | Not equal |
| **AND** | Both |
| **OR** | Either |
| **NOT** | Not |

Have a look at the algorithm below that checks if a percentage mark is valid and whether it is a pass or a fail. This makes use of two **IF** statements; the second **IF** statement is part of the first **ELSE** path. This is called a **nested IF**.

**Find out more**

Programming is covered in Chapter 8. When you have started your programming, write and test a program for this algorithm.

```
OUTPUT "Please enter a mark "
INPUT PercentageMark
IF PercentageMark < 0 OR PercentageMark > 100
  THEN
    OUTPUT "Invalid Mark"
  ELSE
    IF PercentageMark > 49
      THEN
        OUTPUT "Pass"
      ELSE
        OUTPUT "Fail"
    ENDIF
ENDIF
```

A rejected percentage mark must be either less than zero or greater than 100

This is a nested IF statement, shown clearly by the use of a second level of indentation. The percentage mark is only tested if it is in the correct range

## Activity 7.4

Re-write the algorithm to check for a mark between 0 and 20 and a pass mark of 10.

### CASE OF ... OTHERWISE ... ENDCASE

For a **CASE** statement the value of the variable decides the path to be taken. Several values are usually specified. **OTHERWISE** is the path taken for all other values. The end of the statement is shown by **ENDCASE**.

Have a look at the algorithm below that specifies what happens if the value of **Choice** is **1**, **2**, **3** or **4**.

```
CASE OF Choice
  1 : Answer ← Num1 + Num2
  2 : Answer ← Num1 - Num2
  3 : Answer ← Num1 * Num2
  4 : Answer ← Num1 / Num2
  OTHERWISE OUTPUT "Please enter a valid choice"
ENDCASE
```

## Activity 7.5

Use a **CASE** statement to display the day of the week if the variable **DAY** has a whole number value between **1** and **7** inclusive and an error message otherwise.

### The pseudocode for iteration

When some actions performed as part of an algorithm need repeating this is called iteration. Loop structures are used to perform the iteration.

Pseudocode includes these three different types of loop structure:

| | |
|---|---|
| A set number of repetitions | **FOR … TO … NEXT** |
| A repetition, where the number of repeats is not known, that is completed at least once: | **REPEAT … UNTIL** |
| A repetition, where the number of repeats is not known, that may never be completed: | **WHILE … DO … ENDWHILE** |

All types of loops can all perform the same task, for example displaying ten stars:

```
FOR Counter ← 1 TO 10
    OUTPUT "*"
NEXT Counter
```

A FOR … NEXT loop

```
Counter ← 0
REPEAT
    OUTPUT "*"
    Counter ← Counter + 1
UNTIL Counter > 9
```

A REPEAT … UNTIL loop

```
Counter ← 0
WHILE Counter < 10 DO
    OUTPUT "*"
    Counter ← Counter + 1
ENDWHILE
```

A WHILE … DO … ENDWHILE loop

As you can see, the **FOR … TO … NEXT** loop is the most efficient way for a programmer to write this type of task as the loop counter is automatically managed.

**FOR … TO … NEXT loops**

A variable is set up, with a start value and an end value, this variable is incremented in steps of one until the end value is reached and the iteration finishes. The variable can be used within the loop so long as its value is not changed. This type of loop is very useful for reading values into lists with a known length.

**Link**

For more on arrays see Chapter 8.

```
FOR Counter ← 1 TO 10
    OUTPUT "Enter Name of Student "
    INPUT StudentName[Counter]
NEXT
```

Counter starts at 1 and finishes at 10

Array (see Chapter 8) items StudentName[1] to StudentName[10] have data input

### REPEAT ... UNTIL loop

This loop structure is used when the number of repetitions/iterations is not known and the actions are repeated **UNTIL** a given condition becomes true. The actions in this loop are always completed at least once. This is a post-condition loop as the test for exiting the loop is at the end of the loop.

```
Total ← 0
Mark ← 0
REPEAT
    Total ← Total + Mark
    OUTPUT "Enter value for mark, -1 to finish "
    INPUT Mark
UNTIL Mark = -1
```

Variables `Total` and `Mark` are both initialised to zero.

At least one mark is entered.

### WHILE ... DO ... ENDWHILE loop

This loop structure is used when the number of repetitions/iterations is not known and the actions are only repeated **WHILE** a given condition is true. If the **WHILE** condition is untrue then the actions in this loop are never performed. This is a pre-condition loop as the test for exiting the loop is at the beginning of the loop.

```
Total ← 0
OUTPUT "Enter value for mark, -1 to finish "
INPUT Mark
WHILE Mark <> -1 DO
    Total ← Total + Mark
    OUTPUT "Enter value for mark, -1 to finish"
    INPUT Mark
ENDWHILE
```

Only the variable `Total` is initialised to zero

Condition tested at start of loop

### The pseudocode for input and output statements

**INPUT** and **OUTPUT** are used for the entry of data and display of information. Sometimes **READ** can be used instead of **INPUT** but this is usually used for reading from files – see Chapter 8. Also, **PRINT** can be used instead of **OUTPUT** if a hard copy is required.

**INPUT** is used for data entry; it is usually followed by a variable where the data input is stored, for example:

```
INPUT Name
```

```
INPUT StudentMark
```

**OUTPUT** is used to display information either on a screen or printed on paper; it is usually followed by a single value that is a string or a variable, or a list of values separated by commas, for example:

```
OUTPUT Name
```

```
OUTPUT "Your name is ", Name
```

```
OUTPUT Name1, "Ali", Name3
```

# 7.3 Explaining the purpose of an algorithm

An algorithm sets out the steps to complete a given task. This is usually shown as a flowchart or pseudocode, so that the purpose of the task and the processes needed to complete it are clear to those who study it.

You will be able to practise this skill as you become more familiar with writing and finding and correcting errors in algorithms.

## Example 1: Output an alarm sound

The purpose of the following pseudocode is to output the alarm sound at the appropriate time. The processes are: waiting 10 seconds, getting the current time, checking the current time with the alarm time, and outputting the alarm sound when the times match.

```
REPEAT
    Wait (10)
    Get (Time)
UNTIL Time = Alarm_Time
OUTPUT AlarmSound
```

Waits for 10 seconds

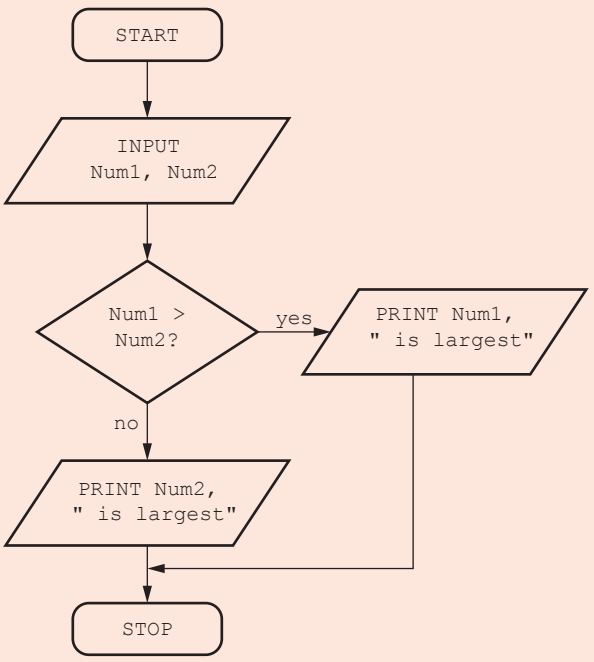
Get current time from the system clock

## Activity 7.6

Have a look at the flowchart and pseudocode below:

- identify the purpose of the algorithm that they both represent
- identify the processes included in the algorithm.

What would be output if the numbers 7 and 18 were input?




```
INPUT Num1, Num2
IF Num1 > Num2
    THEN PRINT NUM1, " is largest"
    ELSE PRINT NUM2, " is largest"
ENDIF
```

▲ **Figure 7.11** Flowchart and pseudocode

## Activity 7.17

Rewrite the pseudocode so it works for every set of numbers like the flowchart above.

Test your pseudocode algorithm with this set of test data:

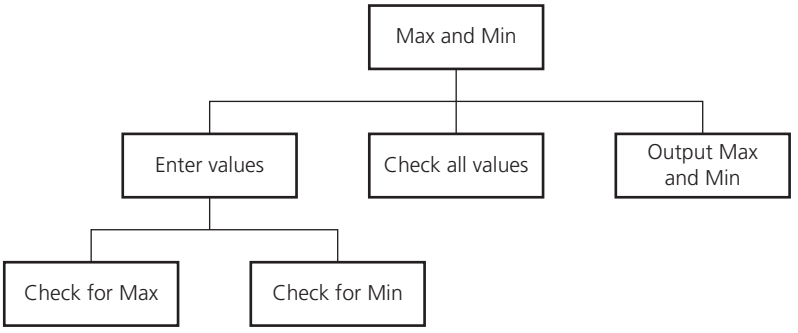
-97, 12390, 0, 77, 359, -2, -89, 5000, 21, 67

# 7.9 Writing and amending algorithms

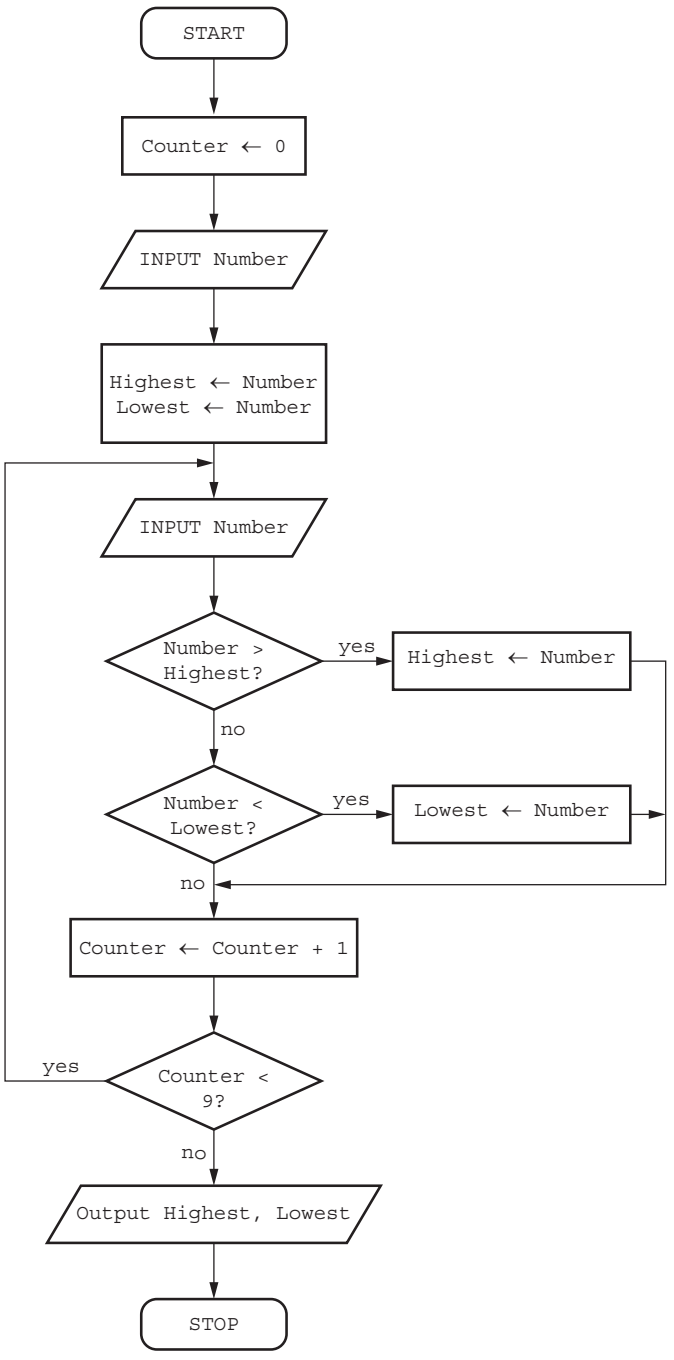
There are a number of stages when producing an algorithm for a given problem:

1. Make sure that the problem is clearly specified – the purpose of the algorithm and the tasks to be completed by the algorithm.
2. Break the problem down in to sub-problems; if it is complex, you may want to consider writing an algorithm for each sub-problem. Most problems, even the simplest ones can be divided into:
   - Set up processes
   - Input
   - Processing of data
   - Permanent storage of data (if required)
   - Output of results.
3. Decide on how any data is to be obtained and stored, what is going to happen to the data and how any results are going to be displayed.
4. Design the structure of your algorithm using a structure diagram.
5. Decide on how you are going to construct your algorithm, either using a flowchart or pseudocode. If you are told how to construct your algorithm, then follow the guidance.
6. Construct your algorithm, making sure that it can be easily read and understood by someone else. Precision is required when writing algorithms, just as it is when writing program code. This involves setting it out clearly and using meaningful names for any data stores. Take particular care with conditions used for loops and selection, for example 'Counter >= 10' rather than 'Counter ten or over'. The algorithms that you have looked at so far in this chapter were not designed with readability in mind because you needed to work out what the problem being solved was.
7. Use several sets of test data (Normal, Abnormal and Boundary) to dry run your algorithm and show the results in trace tables, to enable you to find any errors.
8. If any errors are found, correct them and repeat the process until you think that your algorithm works perfectly.

Have a look at this structure diagram and flowchart for the algorithm to select the largest, Max, and smallest, Min, numbers from a list of ten numbers. This time the flowchart is more easily readable than the structure chart:




▲ **Figure 7.18** Structure chart for Max and Min




▲ **Figure 7.19** A more easily understandable flowchart for Max and Min

## Example 1: Writing algorithms in pseudocode

Tickets are sold for a concert at $20 each. If 10 tickets are bought then the discount is 10%, if 20 tickets are bought the discount is 20%. No more than 25 tickets can be bought in a single transaction.

**a** Use pseudocode to write the algorithm to calculate the cost of buying a given number of tickets.

**b** Explain how you would test your algorithm.

**a**

```
REPEAT
  OUTPUT "How many tickets would you like to buy? "
  INPUT NumberOfTickets
UNTIL NumberOfTickets > 0 AND NumberOfTickets < 26
IF NumberOfTickets < 10
  THEN
    Discount ← 0
  ELSE
    IF NumberOfTickets < 20
      THEN
        Discount ← 0.1
      ELSE
        Discount ← 0.2
    ENDIF
ENDIF
Cost ← NumberOfTickets * 20 * (1 - Discount)
PRINT "Your tickets cost ", Cost
```

**b** I would use test data with values of:

| | |
|---|---|
| **0, 26,** | Expected results rejected |
| **1, 25,** | Expected results **20, 400** |
| **9, 10,** | Expected results **180, 180** |
| **19, 20,** | Expected results **342, 320** |

## Activity 7.18

For the test data given in Example 1, identify the type of test data used and suggest some more test data and dry run the algorithm.

## ? Example 2: Writing algorithms in pseudocode

A school with 600 students wants to produce some information from the results of the four standard tests in Maths, Science, English and IT. Each test is out of 100 marks. The information output should be the highest, lowest and average mark for each test and the highest, lowest and average mark overall. All the marks need to be input.

**a** Use pseudocode to write the algorithm to complete this task.

**b** Explain how you would test your algorithm.

**a**

Comments are used to make the algorithm more understandable

```
// initialisation of overall counters
OverallHighest ← 0
OverallLowest ← 100
OverallTotal ← 0
FOR Test ← 1 TO 4  // outer loop for the tests
```

```
// initialisation of subject counters
  SubjectHighest ← 0
  SubjectLowest ← 100
  SubjectTotal ← 0
  CASE OF Test
    1 : SubjectName ← "Maths"
    2 : SubjectName ← "Science"
    3 : SubjectName ← "English"
    4 : SubjectName ← "IT"
  ENDCASE
  FOR StudentNumber ← 1 TO 600  // inner loop for the
students
    REPEAT
      OUTPUT "Enter Student", StudentNumber, "'s mark for
", SubjectName
      INPUT Mark
    UNTIL Mark < 101 AND Mark > -1
    IF Mark < OverallLowest THEN OverallLowest ← Mark
    IF Mark < SubjectLowest THEN SubjectLowest ← Mark
    IF Mark > OverallHighest THEN OverallHighest ← Mark
    IF Mark > SubjectHighest THEN SubjectHighest ← Mark
    OverallTotal ← OverallTotal + Mark
    SubjectTotal ← SubjectTotal + Mark
  NEXT StudentNumber
  SubjectAverage ← SubjectTotal / 600
  OUTPUT SubjectName
  OUTPUT "Average mark is ", SubjectAverage
  OUTPUT "Highest Mark is ", SubjectHighest
  OUTPUT "Lowest Mark is ", SubjectLowest
NEXT Test
OverallAverage ← OverallTotal / 2400
OUTPUT "Overall Average is ", OverallAverage
OUTPUT "Overall Highest Mark is ", OverallHighest
OUTPUT "Overall Lowest Mark is ", OverallLowest
```

**b** For the algorithm to be tested by dry running, I would reduce the number of students to 5 and the number of subjects to 2.

**Find out more**

Programming is covered in Chapter 8. When you have started your programming, write and test programs for Examples 1 and 2.

More practice on writing algorithms will be given in Chapter 8.

## Activity 7.19

1 Identify the changes you would need to make to the algorithm in Example 2 to reduce the number of students to 5 and the number of subjects to 2.

Identify the test data needed to test Example 2 with the reduced number of students and subjects.

2 With the set of test data you have chosen set up, complete a trace table so that you can compare your expected results with the actual results when you dry run the algorithm.

## Activity 7.20

1 Write pseudocode to input ten positive numbers and find the total and the average.

2 Write pseudocode to input any number of positive numbers and find the total and the average. The user should enter '-1' when they have finished entering their list of positive numbers.

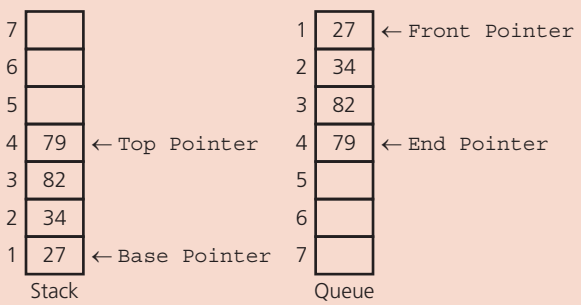
3 Explain why you chose the loop structure for each task.

## Extension

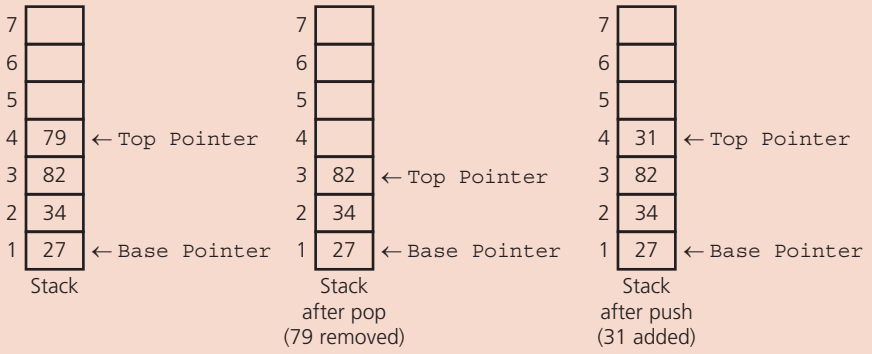
For those students interested in studying computer science at A Level, the following section is an introduction to the use of Abstract Data Types (ADTs) to store data in stacks and queues.

An **ADT** is a collection of data and a set of operations on that data. For example, a stack includes the items held on the stack and the operations to add an item to the stack (push) or remove an item from the stack (pop):

- **stack** – a list containing several items operating on the Last In First Out (LIFO) principle. Items can be added to the stack (push) and removed from the stack (pop). The first item added to a stack is the last item to be removed from the stack.
- **queue** – a list containing several items operating on the First In First Out (FIFO) principle. Items can be added to the queue (enqueue) and removed from the queue (dequeue). The first item added to a queue is the first item to be removed from the queue.

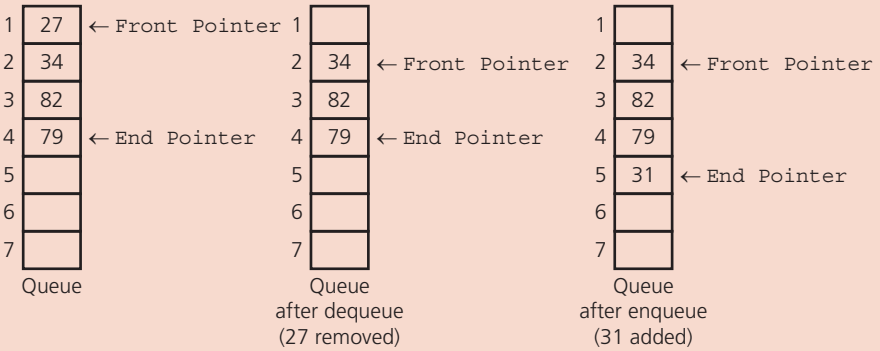



▲ **Figure 7.20** In both of these examples 27 was the first item added and 79 the last item added




▲ **Figure 7.21** Stack operations

The value of the **`Base Pointer`** always remains the same during stack operations.




▲ **Figure 7.22** Queue operations

The values of both the **`Front Pointer`** and the **`End Pointer`** change during queue operations.

### Extension activity

Show a stack and pointers after a pop operation. Show a queue and pointers after a dequeue operation.

In this chapter, you have learnt about:

- ✔ the program development life cycle
- ✔ decomposition of systems and problems in sub-systems and sub-problems
- ✔ design and construction of algorithms to solve problems using structure diagrams, flowcharts, and pseudocode
- ✔ explaining the purpose of an algorithm
- ✔ standard methods of solution
- ✔ the need and purpose of validation and verification checks on input data
- ✔ suggesting and applying suitable test data
- ✔ trace tables and dry runs
- ✔ writing, amending and identifying errors in algorithms.